IMPLEMENT CLIENT SCRIPT & UI POLICY (INCIDENT)

ServiceNow Micro Project Documentation

Project Area: Incident Management

Technologies: ServiceNow, UI Policy, UI Policy Actions, Client Scripts

PROJECT DOCUMENTATION

1. Problem Statement

Incident records often require consistent and accurate data entry to ensure effective triage, routing, and resolution. However, relying solely on user awareness and manual checks can lead to incomplete, inconsistent, or incorrect data being submitted.

This can impact reporting accuracy, SLA compliance, and overall service quality. To address this challenge, there is a need to enforce conditional field behavior and validation directly at the user interface level.

Implementing standardized UI Policies and Client Scripts helps control field visibility, mandatory status, and save-time validation, ensuring that Incident records meet defined business rules before they are submitted.

2. Objective

The objective of the Implement Client Script & UI Policy (Incident) project is to demonstrate how ServiceNow client-side controls can be used to enforce data integrity on Incident records.

The implementation showcases how UI Policies and Client Scripts can dynamically make fields mandatory, auto-populate values, control field behavior, and prevent record submission when required conditions are not met.

The solution ensures that Incidents are created with complete and valid information, improving data quality and consistency across the platform.

3. Skills / Technologies

Incident Management

UI Policy

UI Policy Actions

Client Scripts

Form Validation

4. Main Workflow

Incident Created → Impact Checked → UI Policy Applied → Urgency Controlled → Client Script Updates Urgency → Save Validation → List Edit Restricted → Incident Saved

When an Incident has High Impact, the configured UI Policy applies additional field controls. The onChange Client Script automatically sets Urgency to High. The onSubmit Client Script checks whether Assigned To is filled before saving. The onCellEdit Client Script prevents direct State changes from the list view.

5. Task 1 – Create UI Policy on Incident

Activity 1: Create UI Policy

1. Log in to the ServiceNow instance with administrative or configuration access.

2. Navigate to **System UI → UI Policies**.

3. Click **New** to create a new UI Policy record.

4. Enter the following details:

Field	Value
Name	High Impact Control
Table	Incident
Active	true

Condition Field	Impact
Operator	is
Value	1 – High
Reverse if false	true

The condition ensures that the UI Policy is triggered only when the Incident Impact is set to High.

The UI Policy Action also uses the Assignment group field as a controlled field with Mandatory enabled, and Reverse if false is enabled so the policy changes are reverted when the condition is no longer true.

5. Review the configuration and click **Submit** to save the UI Policy.

6. Task 2 – Create UI Policy Action (Urgency)

Activity 1: Configure the Urgency Field

1. Open the **High Impact Control** UI Policy.

2. Scroll to the **UI Policy Actions** related list.

3. Click **New** to create a UI Policy Action.

4. Configure the action as follows:

• Field name: **Urgency**
• Read-only: **true**
• Visible: Leave unchanged.

5. Click **Submit**.

When Impact = High, the Urgency field becomes read-only while remaining visible. When the condition becomes false, the field can revert according to the configured reverse behavior.

Optional verification: Open an Incident, set Impact to High, confirm Urgency becomes read-only, then change Impact to another value and verify the field reverts when applicable.

7. Task 3 – Create onChange Client Script

Activity 1: Create Client Script

1. Navigate to **System UI** → **Client Scripts**.

2. Click **New**.

3. Configure the Client Script:

• Name: **Auto set urgency for high impact**
• Table: **Incident**
• Type: **onChange**
• Field name: **Impact**
• Active: **true**

4. Paste the following script and click Submit.

```
function onChange(control, oldValue, newValue, isLoading) {
if (isLoading || newValue == '') {
return;
}
if (newValue == '1') {
g_form.setValue('urgency', '1');
g_form.addInfoMessage(
'Urgency set to High for High impact incident.'
);
}
}
```

Verification: Open an Incident and change Impact to High. The Urgency field should update automatically.

8. Task 4 – Create onSubmit Client Script (Save Validation)

Activity 1: Create Client Script

1. Navigate to **System UI** → **Client Scripts**.

2. Click **New**.

3. Configure the Client Script:

• Name: **Prevent save if Assigned To missing**
• Table: **Incident**
• Type: **onSubmit**
• Active: **true**

4. The script checks whether Assigned To is empty when Impact is High. If it is empty, the record is prevented from saving and a warning is displayed.

```
function onSubmit() {
if (g_form.getValue('impact') == '1' &&
g_form.getValue('assigned_to') == '') {
g_form.showErrorBox(
'assigned_to',
'Assigned To is mandatory for High impact incidents.'
);
return false;
}
return true;
}
```

5. Click Submit. Verification: Try saving an Incident with High Impact and no Assigned To. The script should prevent the save.

9. Task 5 – Create onCellEdit Client Script

Activity 1: Create Client Script

1. Navigate to **System UI** → **Client Scripts**.

2. Click **New**.

3. Configure the Client Script:

• Name: **Prevent state change via list edit**
• Table: **Incident**
• Type: **onCellEdit**
• Field name: **State**
• Active: **true**

4. Paste the script below and click Submit.

```
function onCellEdit(sysIDs, table, oldValues, newValue, callback) {
alert('State cannot be updated using list editing. Please open the Incident.');
callback(false);
}
```

Verification: Open an Incident list and try to change State directly. The script should block the change.

10. Task 6 – Testing the Configuration

Activity 1: Test Mandatory Enforcement

1. Go to **Incident** → **Create New**.

2. Set Impact to **High**.

3. Leave Assigned To empty.

4. Click Submit.

5. Expected result: The Incident should not be saved. An error should indicate that Assigned To cannot be empty. Other configured UI Policy and Client Script behaviors should also be checked.

Activity 2: Test Successful Save

1. Fill the Assigned To field.

2. Click Submit.

3. Expected result: The Incident should save successfully without errors. Confirm Urgency auto-setting, read-only behavior, and State restrictions.

Activity 3: Reverse Condition Test

1. Open an existing Incident with Impact = High.

2. Change Impact to Medium.

3. Expected result: Assigned To should no longer be mandatory and Urgency should become editable according to the reverse UI Policy behavior.

Activity 4: Test List Edit Blocking

1. Go to **Incident** → **All**.

2. Double-click the State field of an Incident.

3. Expected result: An alert should appear and the State value should remain unchanged.

Activity 5: Test Form-Based Update

1. Open the Incident record.

2. Change State on the Incident form.

3. Click Update.

4. Expected result: The State change should save successfully, while the other configured rules continue to apply.

11. Expected Results

• High Impact Incidents trigger the configured UI Policy.

• Urgency is automatically set to High through the onChange Client Script.

• Urgency becomes read-only when the UI Policy condition is active.

• Assigned To is validated before saving a High Impact Incident.

• Direct State changes from list editing are blocked.

• Form-based State updates can still be performed.

• Reverse UI Policy behavior restores field behavior when the condition is no longer true.

12. Advantages

• Improves Incident data quality and consistency.

• Reduces manual validation by users.

• Dynamically controls field behavior.

• Prevents incomplete Incident submission.

• Provides automatic field updates.

• Controls list-based editing where required.

• Lightweight and suitable for a short-duration micro project.

13. Limitations

• The configuration is focused on client-side Incident form behavior.

• Exact field behavior depends on the ServiceNow Incident configuration and available fields.

• Client Scripts and UI Policies should be tested carefully with the specific ServiceNow instance configuration.

14. Future Enhancements

• Add additional validation rules for other Incident fields.

• Extend conditional behavior to more Incident scenarios.

• Add more automated user messages and guidance.

• Integrate additional business rules where required.

• Add reporting or monitoring for validation-related issues.

15. Conclusion

The Implement Client Script & UI Policy (Incident) micro project demonstrates how UI Policies and Client Scripts can work together to enforce dynamic field behavior, automate updates, and prevent incorrect data submission on Incident forms.

By using onChange, onSubmit, and onCellEdit scripts alongside UI Policy actions, the project ensures fields such as Assigned To and Urgency respond appropriately to changes in Impact while maintaining clean and consistent data.

The solution also enforces mandatory fields and controls user interactions while remaining lightweight, efficient, and practical for a short-duration ServiceNow micro project.

16. Project Structure / Submission Contents

A suitable submission package can contain:

• Project Documentation PDF

• ServiceNow configuration screenshots

• UI Policy configuration screenshot

• UI Policy Action screenshot

• onChange Client Script screenshot

• onSubmit Client Script screenshot

• onCellEdit Client Script screenshot

• Testing/result screenshots

• Final demonstration recording, if required

17. Submission Note

This documentation follows the same task-based structure as the referenced Incident Client Script & UI Policy project material, including problem statement, objective, skills, configuration tasks, scripts, testing, expected results, advantages, limitations, future enhancements, and conclusion.

The configuration names, conditions, field behavior, scripts, and testing steps are based on the referenced project document.